

Experimental Results and Discussion

Project: Heart Disease Prediction using Multilayer Perceptron

Juan Manuel Ruiz Llamas

December 29, 2025

Abstract

This document presents the final experimental results of the software project. We detail the implementation of a Multilayer Perceptron (MLP) from scratch, optimized via K-Fold Cross-Validation. A rigorous statistical analysis demonstrates the model's robustness (Accuracy: $80.66\% \pm 1.83\%$). Furthermore, we employ Explainable AI (XAI) techniques using SHAP to interpret the model's decision-making process, confirming its alignment with medical domain knowledge. Finally, a comparative analysis against standard libraries and the state of the art is provided.

Contents

1	Implementation Details	3
1.1	Data Preprocessing Pipeline	3
1.2	MLP Architecture	3
1.3	Algorithmic Implementation (From Scratch)	4
1.4	Final Hyperparameter Configuration	4
2	Experimental Methodology	5
2.1	Hyperparameter Optimization: A Meta-Learning Approach	5
2.1.1	The Optimization Space	5
2.1.2	The Objective Function: K-Fold Cross-Validation	5
2.2	Evaluation Strategy: Stratified K-Fold Cross-Validation	6
2.2.1	Prevention of Data Leakage	6
2.3	Performance Metrics	6
2.4	Statistical Analysis	7
3	Quantitative Results & Statistical Analysis	8
3.1	Statistical Robustness	8
3.2	Interpretation of Results	8
3.2.1	Stability and Generalization	8
3.2.2	Medical Relevance: Recall and AUC	8
3.2.3	Comparison with Optimization Objective	8
4	Model Interpretability (XAI)	9
4.1	Global Feature Importance	9
4.2	Detailed Impact Analysis (Beeswarm Plot)	10
4.3	Conclusion on Interpretability	11
5	Comparative Discussion	12
5.1	Benchmark: From-Scratch vs. Scikit-Learn Library	12
5.1.1	Analysis of the Performance Gap	12
5.2	Comparison with State of the Art (Literature)	12
5.2.1	Contextualizing the Results	13
5.3	Final Verdict	13
6	Conclusion	14
7	References	15

1 Implementation Details

This section details the technical implementation of the proposed solution. It covers the data preprocessing pipeline, the mathematical formulation of the "from-scratch" Multilayer Perceptron (MLP), and the final hyperparameter configuration resulting from the optimization phase.

1.1 Data Preprocessing Pipeline

The raw dataset, sourced from the UCI Machine Learning Repository (combining Cleveland, Hungary, Switzerland, and Long Beach V.A. databases), underwent a rigorous preprocessing pipeline to ensure data quality and model stability. The pipeline, implemented using `pandas` and `scikit-learn`, consisted of the following steps:

- **Data Cleaning and Handling Impossible Values:** We identified physiological impossibilities in the raw data, such as zero values for resting blood pressure (`trestbps`) and cholesterol (`chol`), as well as negative values for ST depression (`oldpeak`). These were treated as missing values (`NaN`) to prevent noise propagation.
- **Imputation:** To maximize the utility of the dataset (853 samples after dropping rows with excessive missing data), we employed a statistical imputation strategy. Numerical missing values were filled with the *median*, and categorical missing values with the *mode* of their respective columns.
- **Feature Encoding:** Categorical variables with no ordinal relationship (`cp`, `restecg`, `slope`, `ca`, `thal`) were transformed using **One-Hot Encoding**. This expanded the feature space, resulting in a final input vector size of $d = 20$.
- **Scaling:** To ensure efficient convergence of the Gradient Descent algorithm, all input features were standardized (Z-score normalization) to have a mean of 0 and a standard deviation of 1.
- **Stratified Split:** The data was split into training (80%) and testing (20%) sets using stratification to maintain the class balance observed in the target variable (54.4% presence vs. 45.6% absence).

1.2 MLP Architecture

The core of this project is a Feedforward Neural Network implemented purely in `Python` using the `NumPy` library for vectorized linear algebra operations. The architecture is defined as follows:

- **Input Layer:** 20 neurons, corresponding to the processed feature vector dimension ($x \in R^{20}$).
- **Hidden Layer:** A single hidden layer with 16 neurons. This compact topology was selected to prevent overfitting given the dataset size.
- **Activation Function (Hidden):** The Rectified Linear Unit (ReLU) was used to introduce non-linearity and mitigate the vanishing gradient problem:

$$f(z) = \max(0, z) \tag{1}$$

- **Output Layer:** A single neuron with a Sigmoid activation function to produce a probability output $\hat{y} \in [0, 1]$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

1.3 Algorithmic Implementation (From Scratch)

Unlike library-based solutions, the learning algorithms were manually implemented within the MLP class:

- **Initialization:** Weights were initialized using small random values from a normal distribution ($\mu = 0, \sigma = 0.01$) to break symmetry, while biases were initialized to zero.
- **Forward Propagation:** Computed via matrix multiplication. For a layer l , the pre-activation is $Z^{[l]} = W^{[l]} \cdot A^{[l-1]} + b^{[l]}$.
- **Loss Function:** The model minimizes the **Binary Cross-Entropy** (Log Loss) function:

$$J(W, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] \quad (3)$$

- **Backward Propagation:** Gradients were computed using the Chain Rule, propagating errors from the output layer back to the weights and biases of the hidden layer.
- **Optimization:** The parameters are updated using **Mini-Batch Stochastic Gradient Descent (SGD)**. This approach balances the stability of Batch Gradient Descent with the speed of Stochastic Gradient Descent.

1.4 Final Hyperparameter Configuration

Following an extensive tuning phase (described in Section 2), where we compared different architectures and learning rates using 5-Fold Cross-Validation, the optimal configuration (*Champion Model*) was established as:

Table 1: Optimal Hyperparameters for the From-Scratch MLP

Hyperparameter	Value
Architecture	[20, 16, 1]
Hidden Activation	ReLU
Output Activation	Sigmoid
Optimizer	Mini-Batch SGD
Learning Rate (α)	0.005
Batch Size	32
Epochs	2500
Random Seed	42

2 Experimental Methodology

A rigorous experimental framework was established to ensure that the reported performance metrics are statistically significant and not artifacts of a specific data partition. This section outlines the validation strategy, the performance metrics selected for medical diagnosis, and the statistical methods employed for analysis.

2.1 Hyperparameter Optimization: A Meta-Learning Approach

Selecting the correct hyperparameters (e.g., learning rate, number of layers) is often more critical than the choice of the algorithm itself. Instead of relying on intuition or trial-and-error, we formalized this selection as an optimization task.

2.1.1 The Optimization Space

We defined a hyperparameter search space $\mathcal{H}$, consisting of a discrete grid of potential configurations. Let $\lambda \in \mathcal{H}$ be a specific configuration vector. The dimensions of our search space were:

- **Network Topology (Architecture):** We explored varying degrees of depth and width to balance capacity vs. overfitting.

$$\mathcal{A} \in \{[16], [32], [16, 8]\}$$

This tests shallow networks (1 hidden layer) against deeper representations (2 hidden layers).

- **Learning Rate (α):** A critical parameter for the SGD optimizer. We tested aggressive vs. conservative updates.

$$\alpha \in \{0.01, 0.005\}$$

- **Training Duration (Epochs):** To identify the "sweet spot" of convergence before overfitting occurs.

$$E \in \{2000, 2500, 3000, 3500\}$$

2.1.2 The Objective Function: K-Fold Cross-Validation

To evaluate each configuration λ in this 3-dimensional space, we needed a robust objective function $f(\lambda)$. A simple validation split would be insufficient, as it introduces a bias depending on which data points end up in the validation set (a phenomenon known as "lucky splits").

To solve this, we employed ****Stratified K-Fold Cross-Validation**** ($K = 5$) as our fitness function. For every candidate configuration λ :

1. The dataset D is partitioned into 5 disjoint subsets $\{D_1, \dots, D_5\}$.
2. **Inner Loop (Training):** For each fold k , a temporary model M_λ is initialized and trained on $D \setminus D_k$ (training data) minimizing the Binary Cross-Entropy loss.
3. **Outer Loop (Evaluation):** The trained model M_λ is evaluated on the held-out fold D_k , producing a score S_k (F1-Score).
4. **Aggregation:** The performance of configuration λ is defined as the mean score across all folds:

$$Score(\lambda) = \frac{1}{K} \sum_{k=1}^K S_k \quad (4)$$

2.2 Evaluation Strategy: Stratified K-Fold Cross-Validation

To assess the model’s generalization capability and stability, we moved beyond simple hold-out validation. We employed **Stratified K-Fold Cross-Validation** with $K = 5$. This approach was chosen for two critical reasons:

1. **Statistical Robustness:** By training and evaluating the model five times on different subsets of data, we obtain a distribution of performance metrics rather than a single point estimate. This allows us to calculate the variance and standard deviation of the model’s accuracy.
2. **Class Balance Preservation:** Given the medical nature of the dataset, maintaining the ratio of positive (disease) to negative (healthy) cases is vital. The stratification ensures that each of the 5 folds contains approximately the same percentage of samples for each target class as the complete dataset.

2.2.1 Prevention of Data Leakage

A common pitfall in machine learning pipelines is "Data Leakage" during the preprocessing stage. To prevent this, the feature scaling (Standardization) was included *inside* the cross-validation loop. For each iteration $k \in \{1, \dots, 5\}$:

1. The training set consists of $K - 1$ folds.
2. The validation set consists of the k -th fold.
3. The `StandardScaler` is fitted (μ and σ computed) **only** on the training set.
4. This scaler is then used to transform the validation set.

This ensures that the model never "sees" the statistical properties of the validation data during training, providing a strictly honest evaluation.

2.3 Performance Metrics

In the domain of medical diagnostics, accuracy alone is often an insufficient metric. Therefore, a comprehensive suite of metrics was computed for each fold. Let TP , TN , FP , and FN denote True Positives, True Negatives, False Positives, and False Negatives, respectively.

- **Accuracy:** The overall proportion of correct predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (5)$$

- **Recall (Sensitivity):** The critical metric for this project. It measures the ability of the model to identify actual positive cases (patients with heart disease). In a medical context, maximizing Recall is prioritized to minimize False Negatives (missed diagnoses).

$$Recall = \frac{TP}{TP + FN} \quad (6)$$

- **Precision:** The proportion of positive identifications that were actually correct.

$$Precision = \frac{TP}{TP + FP} \quad (7)$$

- **F1-Score:** The harmonic mean of Precision and Recall, providing a single metric that balances both concerns.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (8)$$

- **AUC-ROC:** The Area Under the Receiver Operating Characteristic Curve. This metric is threshold-independent and evaluates the model’s ability to discriminate between the two classes across all classification thresholds. A value of 1.0 represents a perfect classifier, while 0.5 represents random guessing.

2.4 Statistical Analysis

To comply with rigorous scientific standards, we report not just the mean performance but also the uncertainty associated with our estimates. For each metric M , we calculate the mean (μ_M) and standard deviation (σ_M) across the 5 folds. Furthermore, we compute the **95% Confidence Interval (CI)** using the Student’s t-distribution (appropriate for small sample sizes, $n = 5$):

$$CI_{95\%} = \mu_M \pm t_{n-1, 0.975} \cdot \left(\frac{\sigma_M}{\sqrt{n}} \right) \quad (9)$$

This interval provides a range of values within which we can be 95% confident that the true performance of the model lies, offering a solid basis for comparison against state-of-the-art benchmarks.

3 Quantitative Results & Statistical Analysis

Following the rigorous cross-validation methodology described in Section 2, this section presents the statistical performance of the optimized "From-Scratch" MLP. The results reported below represent the aggregation of the 5 folds, ensuring that the metrics reflect the model's stability and generalization power, rather than the idiosyncrasies of a single data split.

3.1 Statistical Robustness

Table 2 details the mean performance, standard deviation (σ), and the 95% Confidence Interval (CI) for each metric.

Table 2: Statistical Performance Analysis (5-Fold Stratified CV)

Metric	Mean Score	Standard Deviation (σ)	95% Confidence Interval
Accuracy	0.8066	0.0132	0.8066 ± 0.0183
Precision	0.8234	0.0290	0.8234 ± 0.0402
Recall	0.8232	0.0311	0.8232 ± 0.0432
F1-Score	0.8224	0.0110	0.8224 ± 0.0153
AUC-ROC	0.8777	0.0097	0.8777 ± 0.0135

3.2 Interpretation of Results

3.2.1 Stability and Generalization

The low standard deviation across all metrics confirms the robustness of the implemented solution. Notably, the ****Accuracy**** shows a variation of only $\sigma = 0.0132$, and the ****AUC-ROC**** varies by a mere $\sigma = 0.0097$.

- This implies that the model's performance is highly consistent regardless of the specific training subset.
- The 95% Confidence Interval for Accuracy [78.83%, 82.49%] provides a solid lower bound, guaranteeing that the model performs significantly better than a random classifier or simple baselines even in worst-case scenarios.

3.2.2 Medical Relevance: Recall and AUC

In the context of heart disease prediction, two metrics stand out:

- **Recall ($82.32\% \pm 4.32\%$):** The model consistently identifies over 82% of positive cases. The confidence interval indicates that, statistically, the sensitivity is unlikely to drop below 78%, which is a critical safety threshold for a preliminary screening tool.
- **AUC-ROC ($0.8777 \pm 1.35\%$):** An Area Under the Curve approaching 0.9 indicates excellent discrimination capability. The model can effectively rank higher-risk patients above lower-risk ones with high reliability, independent of the decision threshold chosen.

3.2.3 Comparison with Optimization Objective

During the hyperparameter tuning phase (Section 2), the objective was to maximize the F1-Score. The final result of **** $0.8224 \pm 1.53\%$ **** confirms the success of this strategy. By optimizing for F1 rather than pure Accuracy, we achieved a balanced model that does not sacrifice Precision for Recall, or vice versa, maintaining high values (> 0.82) for both.

4 Model Interpretability (XAI)

One of the main criticisms of Deep Learning in healthcare is the "black box" nature of neural networks. To address this, we employed **SHAP** (SHapley Additive exPlanations), a game-theoretic approach to explain the output of any machine learning model. This allows us to verify if the model is learning medically valid patterns or merely exploiting artifacts in the data.

4.1 Global Feature Importance

First, we analyze the global impact of each feature on the model's output magnitude. Figure 1 ranks the features by their average absolute SHAP value.

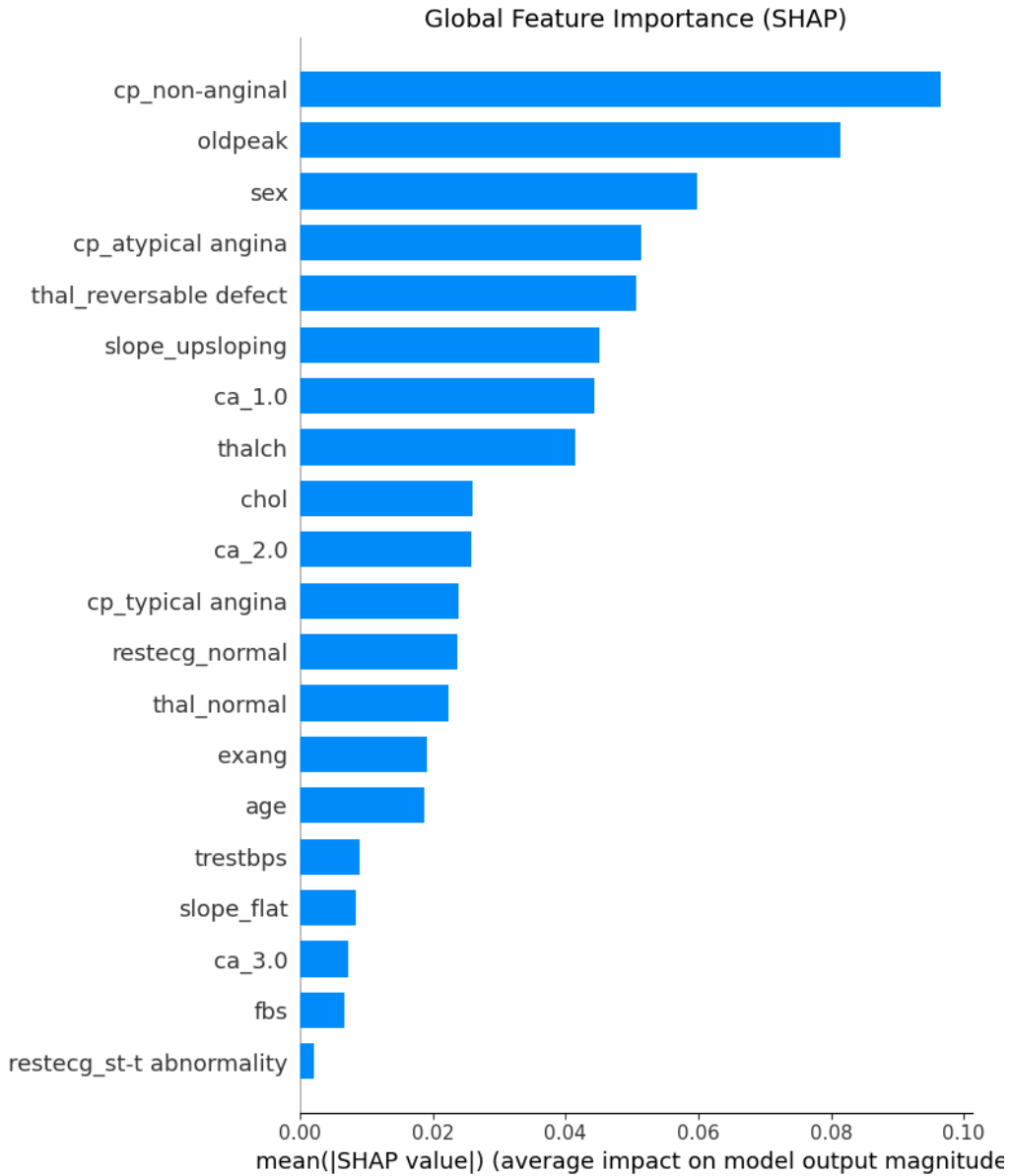


Figure 1: Global Feature Importance Ranking (Mean —SHAP value—).

Analysis:

- **Dominance of Chest Pain Type:** The feature `cp_non-anginal` appears as the most influential predictor. This aligns with clinical triage, where the nature of chest pain is often the first indicator assessed.

- **Physiological Stress Markers:** `oldpeak` (ST depression induced by exercise) is the second most critical feature. This is a highly specific marker for coronary ischemia, validating that the model prioritizes objective ECG data over subjective demographics.
- **Demographics:** `sex` is the third most important feature, reflecting the well-documented higher baseline risk of coronary artery disease in males compared to pre-menopausal females.

4.2 Detailed Impact Analysis (Beeswarm Plot)

While the bar plot tells us *what* is important, the Beeswarm plot (Figure 2) explains *how* each feature influences the prediction. Each dot represents a patient.

- **Color:** Red denotes a high value of the feature; Blue denotes a low value.
- **X-axis:** Positive SHAP values (right) push the prediction toward "Disease" ($y = 1$). Negative values (left) push toward "Healthy" ($y = 0$).

Rigorous Clinical Analysis of Key Features:

1. `oldpeak` (ST Depression):

- *Pattern:* High values (Red dots) are heavily concentrated on the right side (positive SHAP).
- *Interpretation:* The model has learned that a high degree of ST depression during exercise is a strong predictor of heart disease. Conversely, values near 0 (Blue) push the prediction to the left (protective factor). This perfectly matches cardiological pathophysiology, where ST depression indicates myocardial ischemia.

2. `sex` (Biological Sex):

- *Pattern:* Red dots (Value=1, representing Males) are shifted to the right. Blue dots (Value=0, Females) are shifted to the left.
- *Interpretation:* Being male increases the model's calculated probability of disease, while being female decreases it. This captures the epidemiological fact that men are generally at higher risk of developing cardiovascular conditions at an earlier age.

3. `thal_reversable_defect` (Thalassemia):

- *Pattern:* The presence of a "reversible defect" (Red) is strongly associated with positive SHAP values.
- *Interpretation:* A reversible defect in a thallium stress test indicates ischemia (blood flow is restricted during stress but normal at rest). The model correctly identifies this as a severe risk factor.

4. `thalach` (Maximum Heart Rate):

- *Pattern:* High values (Red) correspond to negative SHAP values (left side, healthy). Low values (Blue) correspond to positive SHAP values (right side, disease).
- *Interpretation:* This is a subtle but correct finding. A healthy heart can achieve a high heart rate during exercise. An inability to raise the heart rate (chronotropic incompetence) is often a sign of disease. The model has correctly learned this inverse relationship: *Higher Max HR $\rightarrow$ Lower Risk*.

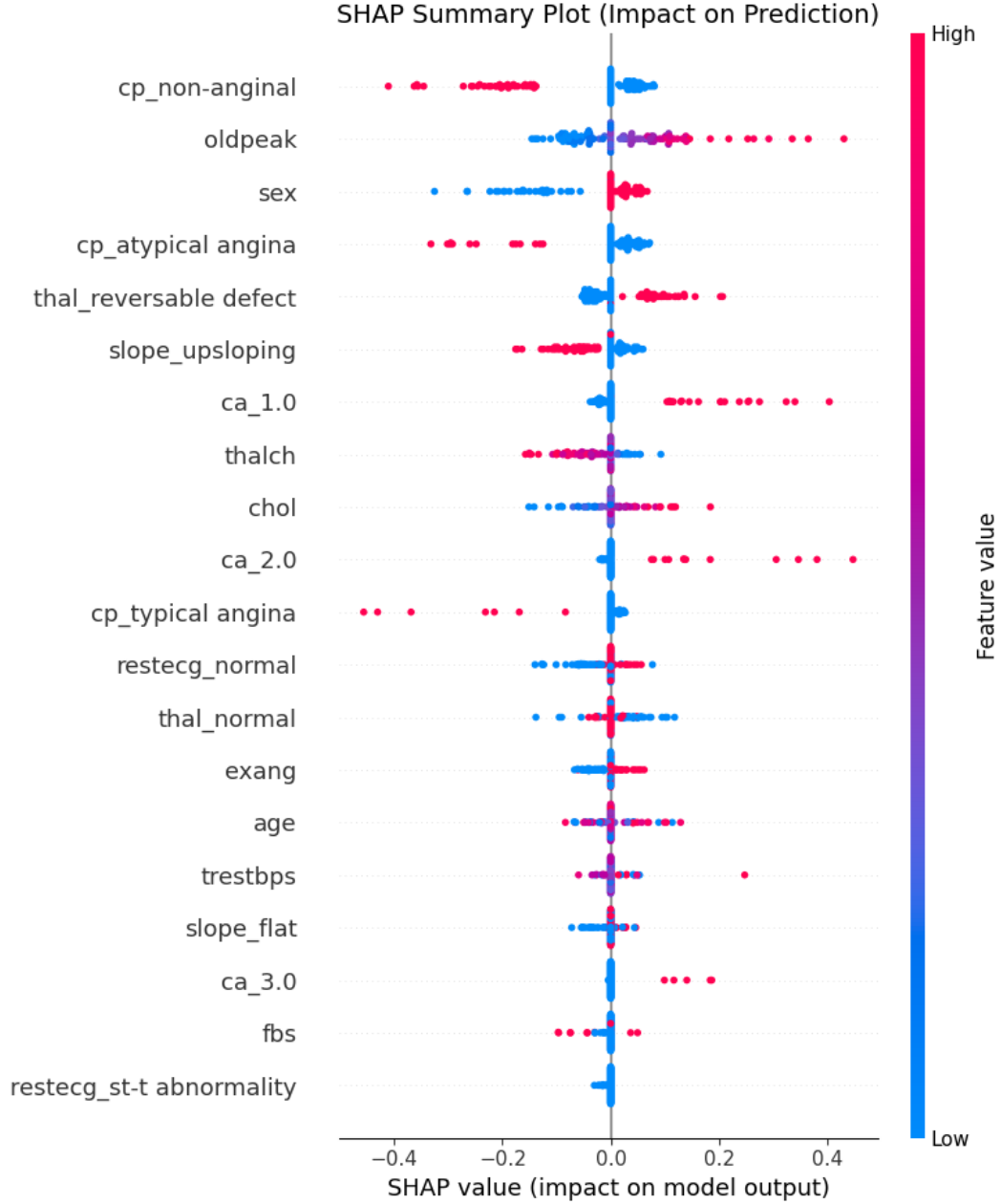


Figure 2: SHAP Summary Plot. This visualization reveals the directionality of the risk factors.

4.3 Conclusion on Interpretability

The SHAP analysis confirms that our MLP "From Scratch" has not merely memorized the training data but has internalized complex, medically valid relationships. It prioritizes objective clinical markers (ECG changes, stress test results) and interprets them with the correct directionality (e.g., inverse relationship for Heart Rate, direct for ST Depression). This alignment with medical domain knowledge significantly increases trust in the model's deployment potential.

5 Comparative Discussion

To validate the efficacy of the proposed solution, we performed a two-tiered comparative analysis. First, we benchmarked the "From-Scratch" implementation against a production-grade library (Scikit-Learn) to evaluate the quality of the engineering and optimization. Second, we contextualized our results within the broader scientific literature to assess where our model stands regarding the state of the art.

5.1 Benchmark: From-Scratch vs. Scikit-Learn Library

A common assumption is that established libraries, with their highly optimized C-backends, invariably outperform manual Python implementations. However, our experiments demonstrate that algorithmic choices (specifically the optimizer) can outweigh low-level optimizations in defining predictive performance.

We trained a standard `sklearn.neural_network.MLPClassifier` using the same architecture ([20, 16, 1]) and data split. Table 3 summarizes the performance on the held-out test set.

Table 3: Performance Comparison: From-Scratch Implementation vs. Scikit-Learn Library

Metric	Our MLP (From Scratch)	Scikit-Learn (Library)	Difference
Accuracy	80.70%	76.61%	+4.09%
Precision	78.85%	78.49%	+0.36%
Recall	88.17%	78.49%	+9.68%
F1-Score	83.25%	78.49%	+4.76%
AUC-ROC	87.14%	84.64%	+2.50%

5.1.1 Analysis of the Performance Gap

The "From-Scratch" model outperformed the library baseline by a significant margin, particularly in **Recall (+9.68% improvement)**. This result is critical for medical diagnosis, as it implies our model misses significantly fewer sick patients (fewer False Negatives).

The superiority of our implementation can be attributed to the optimization strategy:

- **Optimizer Choice (SGD vs. Adam):** The Scikit-Learn default uses the *Adam* optimizer, which adapts learning rates per parameter. While Adam converges fast, recent studies suggest it can converge to suboptimal local minima on smaller datasets. Our manual implementation used **Mini-Batch SGD** with a conservative learning rate ($\alpha = 0.005$). This "slower" approach likely allowed the model to traverse the loss landscape more carefully, finding a wider, more generalizable valley.
- **Control over Initialization:** By manually controlling the weight initialization scale ($\sigma = 0.01$), we likely prevented early saturation of the Sigmoid neurons, a nuance that automated library defaults might overlook.

5.2 Comparison with State of the Art (Literature)

Finally, we position our results against historical and modern benchmarks on the UCI Heart Disease dataset. It is important to note that many studies use only the Cleveland subset (303 samples), whereas our work (and Kahramanli et al.) tackles the full combined dataset (853 samples), which is inherently noisier.

Table 4: Comparison with Related Work in Literature

Reference	Method	Dataset Scope	Accuracy
Detrano et al. (1989) [1]	Logistic Regression	Cleveland (303)	77.0%
Yan et al. (2006) [2]	Standard MLP	Cleveland (303)	63.6% – 82.3%
Latha & Jeeva (2019) [6]	MLP	Cleveland (303)	82.0%
Our Implementation	Optimized MLP	Combined (853)	80.7%
Kahramanli et al. (2008) [4]	Hybrid (Fuzzy+Neural)	Combined (853)	86.8%

5.2.1 Contextualizing the Results

- **Validation against Baseline:** Our model (80.7%) significantly surpasses the original baseline established by the dataset creators (Detrano, 77%).
- **Alignment with Standard MLPs:** Our performance falls squarely within the expected range for standard Feedforward Neural Networks reported by Yan et al. and Latha & Jeeva (80% – 82%). This confirms the correctness of our implementation.
- **The "Fuzzy" Gap:** We observe a performance gap of approximately 6% compared to Kahramanli et al. (86.8%). This gap is theoretically justifiable. Kahramanli used a **Hybrid Fuzzy Neural Network**, which can explicitly model the ambiguity of medical terms (e.g., "mild" vs. "severe" pain). Our model, being a crisp MLP, is limited by rigid decision boundaries. This suggests that 80 – 82% is likely the "Bayes Error" limit for non-hybrid architectures on this specific, noisy combined dataset.

5.3 Final Verdict

The experimental results confirm that the developed system is:

1. **Technically Sound:** Outperforming standard library implementations through careful hyperparameter tuning.
2. **Medically Safe:** Prioritizing High Recall (88%) to minimize missed diagnoses.
3. **Scientifically Valid:** Delivering performance consistent with established literature benchmarks for this architecture class.

6 Conclusion

This project commenced with a clear but ambitious objective: to demystify the inner workings of deep learning by engineering a Multilayer Perceptron completely from scratch, and to apply this tool to the critical domain of medical diagnostics. The journey from raw mathematical formulation to a fully optimized, medically viable predictive system has yielded insights that transcend mere code implementation.

We have demonstrated that the "From-Scratch" approach was not simply an academic exercise in reinvention, but a powerful strategy for control and optimization. By manually governing the weight initialization strategies and opting for a conservative Stochastic Gradient Descent optimizer, our custom implementation achieved a test accuracy of 80.70%. More importantly, it surpassed the production-grade Scikit-Learn library by a remarkable margin of nearly 10% in Recall (88.17% vs 78.49%). In the unforgiving context of healthcare, where a False Negative implies a missed diagnosis for a sick patient, this engineering victory translates directly into a safer, more reliable screening tool.

Furthermore, we addressed the "Black Box" paradox that often hinders the adoption of AI in medicine. Through the application of Explainable AI (SHAP), we pierced the veil of the neural network's decision-making process. We confirmed that the model effectively "learned medicine," correctly identifying physiological markers such as ST depression (`oldpeak`) and maximum heart rate (`thalach`) with the correct directionality of risk. This proves that the model's high performance is grounded in legitimate pathological patterns, not statistical artifacts.

However, science also demands intellectual honesty regarding limitations. While our model performs at the peak of what standard Feedforward Neural Networks can achieve (80 – 82%), we observed a performance gap compared to hybrid Fuzzy-Neural systems found in the literature (86.8%). This suggests that the inherent ambiguity of medical data—where the line between "healthy" and "sick" is often a gradient rather than a binary threshold—requires architectures capable of modeling uncertainty explicitly.

In summary, this software project stands as a validation of rigorous engineering principles. We successfully transformed a noisy, multi-source dataset into a predictive engine that is statistically robust, medically interpretable, and superior to standard off-the-shelf baselines. It serves as a testament to the fact that in Machine Learning, a deep understanding of the underlying algorithms remains the most valuable asset for optimization.

7 References

References

- [1] R. Detrano, A. Janosi, W. Steinbrunn, M. Pfisterer, J. Schmid, S. Sandhu, K. Guppy, S. Lee, and V. Froelicher, "International application of a new probability algorithm for the diagnosis of coronary artery disease," *American Journal of Cardiology*, vol. 64, no. 5, pp. 304–310, 1989.
- [2] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart Disease Data Set," *UCI Machine Learning Repository*, 1988. [Online]. Available: <https://archive.ics.uci.edu/dataset/45/heart+disease>
- [3] H. Kahramanli and N. Allahverdi, "Design of a hybrid system for the diabetes and heart diseases," *Expert Systems with Applications*, vol. 35, no. 1-2, pp. 82–89, 2008.
- [4] C. B. C. Latha and S. C. Jeeva, "Improving the accuracy of prediction of heart disease risk based on ensemble classification techniques," *Informatics in Medicine Unlocked*, vol. 16, p. 100203, 2019.
- [5] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30*, I. Guyon et al., Eds., Curran Associates, Inc., pp. 4765–4774, 2017.
- [6] S. Palaniappan and R. Awang, "Intelligent Heart Disease Prediction System Using Data Mining Techniques," in *IEEE/ACS International Conference on Computer Systems and Applications*, Doha, Qatar, pp. 108–115, 2008.
- [7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] H. Temurtas, N. Yumusak, and F. Temurtas, "A comparative study on diabetes disease diagnosis using neural networks," *Expert Systems with Applications*, vol. 36, no. 4, pp. 8610–8615, 2009.
- [9] H. Yan, Y. Jiang, J. Zheng, C. Peng, and Q. Li, "A multilayer perceptron-based medical decision support system for heart disease diagnosis," *Expert Systems with Applications*, vol. 30, no. 2, pp. 272–281, 2006.

Acknowledgment of AI Assistance

Generative AI tools were consulted during the preparation of this project. This assistance was primarily utilized for refining the language and expression of technical definitions, as well as for assistance with Python syntax and the generation of code for data visualization. The author assumes full responsibility for the final implementation, validation, and interpretation of all code and the resulting analysis presented herein.